



www.tresdefebrero.gov.ar/tecno3f

#ProgramáTuFuturo





Municipalidad de
Tres de Febrero



Programá
tu futuro



PYTHON INTERMEDIO

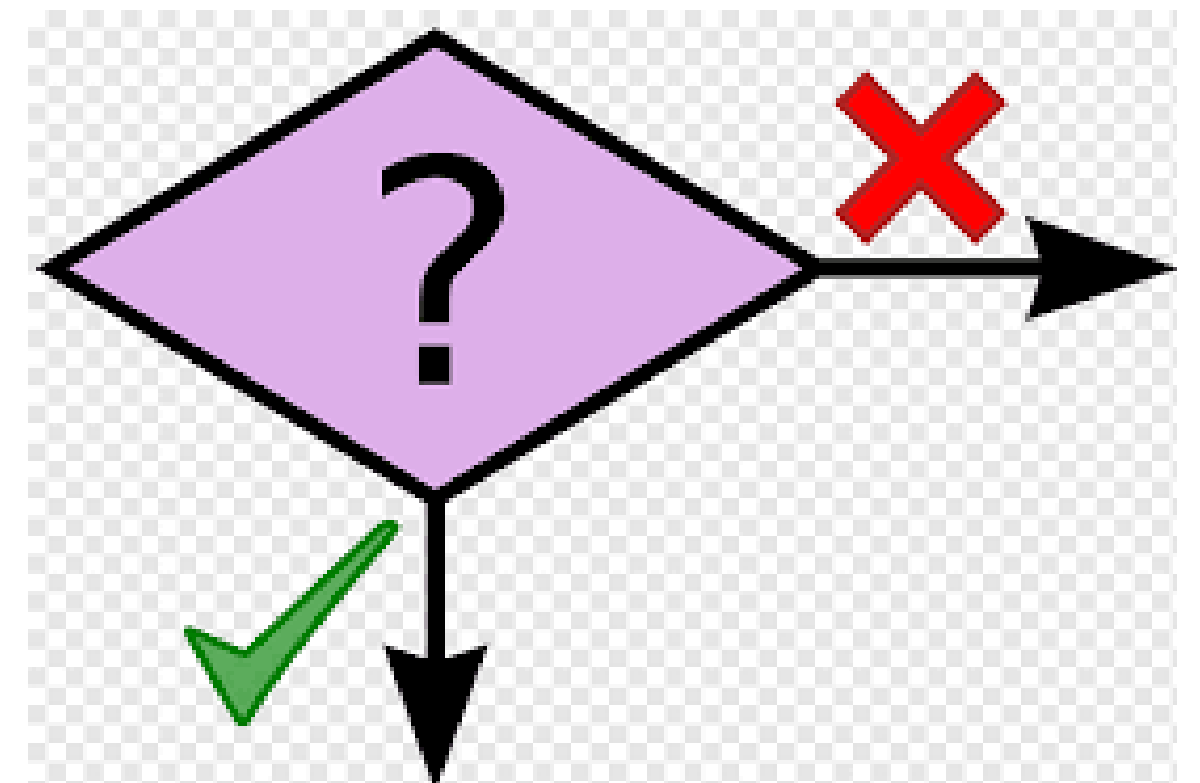
¡Les damos la bienvenida!



¿QUÉ ES?

El operador ternario, también conocido como expresión condicional, es una herramienta muy útil en Python que te permite realizar una comparación y obtener un resultado en una sola línea de código.

Se asemeja a la estructura de un "if-else" pero de forma concisa..



Estructura

es una estructura bastante sencilla

Valor Verdadero **if** Condición **else** Valor Falso

Ejemplos

Como podemos ver, los ternarios se pueden ejecutar en el mismo **print**

```
edad = 18

print("Es mayor de edad" if edad >= 18 else "Es menor de edad")
```

También podemos guardar el resultado en una variable para mostrarlo cuando necesitemos

```
mostrar_edad = "Es mayor de edad" if edad >= 18 else "Es menor de edad"

print(mostrar_edad)
```

Anidando

Al igual que los condicionales, podemos anidar los operadores ternarios

```
a = 1
b = 3
c = 2

numero_mayor = a if a > b and a > c else b if b > c else c

print(numero_mayor)
```

Que valor se mostrara por consola ?

Manden sus respuestas al chat

Respuesta

Para saber la repuesta interpretamos la sentencia anterior asi:

1 si 1 es mayor a 3 y 1 mayor a 2 , sino , 3 si 3 es mayor a 2 , sino, 2

El valor que se mostrara es 3

1 si 1

* ARGS



Programá
tu futuro



Municipalidad de
Tres de Febrero

¿QUÉ ES?

El Los *ARGS (que no necesariamente deben llamarse así), permiten pasar un número variable de argumentos.

¿COMO USARLO?

Lo mas importante es respetar el *

Veamos un ejemplo

Ejemplo

```
def test_var_args(f_arg, *argv):  
    print("primer argumento normal:", f_arg)  
    print(argv) #esto regresa una tupla  
    for arg in argv:  
        print("argumentos de *argv:", arg)  
  
test_var_args('python', 'foo', 'bar')
```

***** KWARGS**



Programá
tu futuro



Municipalidad de
Tres de Febrero

¿QUÉ ES?

Al igual que con los *ARGS lo **KWARGS permiten pasar un numero variable de argumentos, con la diferencia , que en este caso se pasa key y value.

¿COMO USARLO?

Lo mas importante es respetar el **

Veamos un ejemplo

Ejemplo

```
def saludame(**kwargs):  
    print(kwargs) #esto nos regresa un diccionario  
    for key, value in kwargs.items():  
        print("{0} = {1}".format(key, value))  
  
saludame(nombre = "Gabriel", apellido = "Gomez", provincia = "Buenos Aires")
```

¿Como usarlos?

Dependerá mucho de los requisitos de tu programa, pero uno de los usos más comunes es para crear decoradores para funciones. También puede ser usado para **monkey patching**, lo que significa modificar código en tiempo de ejecución.

Considera por ejemplo que tienes una clase con una función llamada `get_info` que llama a una API que devuelve una determinada respuesta

DECORADORES



Programá
tu futuro



Municipalidad de
Tres de Febrero

¿QUÉ ES?

Un decorador es una función que recibe otra función como argumento, le añade o modifica comportamiento (antes o después), y retorna una nueva función modificada sin alterar su código original.

Principio Clave

En Python las funciones son objetos de primera clase (First-class citizens): se pueden guardar en variables, pasar como parámetros y retornar desde otras funciones.

Veamos la Sintaxis

Sintaxis

Sintaxis moderna con azúcar sintáctico (@):

```
@mi_decorador  
def saludar():  
    print("¡Hola mundo!")
```

Es exactamente equivalente a escribir:
saludar = mi_decorador(saludar)

¿Por qué los Decoradores Necesitan `*args` y `**kwargs`?

Un decorador reutilizable debe ser capaz de decorar cualquier función: funciones sin argumentos, funciones con 2 números, o funciones con 5 claves opcionales.

```
def decorador_universal(funcion_original):  
    def envoltorio(*args, **kwargs):  
        # 1. Código previo a la ejecución  
        print("--> [PRE]: Preparando ejecución...")  
        # 2. Invocamos la función original reenviando todo sin pérdidas  
        resultado = funcion_original(*args, **kwargs)  
        # 3. Código posterior a la ejecución  
        print("--> [POST]: Finalizado con éxito.")  
        return resultado  
    return envoltorio
```

¡MUCHAS GRACIAS!



Programá
tu futuro



Municipalidad de
Tres de Febrero